Equality Joins

```
SELECT  *
FROM    Users U, GroupMembers GM
WHERE   U.uid = GM.uid
```

	uid	uname	experience	age
$p1_u$	123	Dora	2	13
	132	Bug	8	60
$p2_u$	267	Sakura	7	15
	111	Cyphon	8	35

	uid	gid	stars
$p1_g$	123	G1	2
	132	G1	5
$p2_g$	132	G2	3
	132	G3	1
$p3_g$	123	G2	4
	111	G4	2

Join Cardinality Estimation

- | Users ⋈ GroupMembers | = ?
  - Join attribute is primary key for Users, foreign key in GroupMember
  - Each GroupMember tuple matches exactly with one Users tuple
  - Result: |GroupMembers|
- | Users X GroupMembers | = ?
  - Result: |Users| * |GroupMembers|
  - Cross product is always the product of individual relation sizes
- For other joins more difficult to estimate

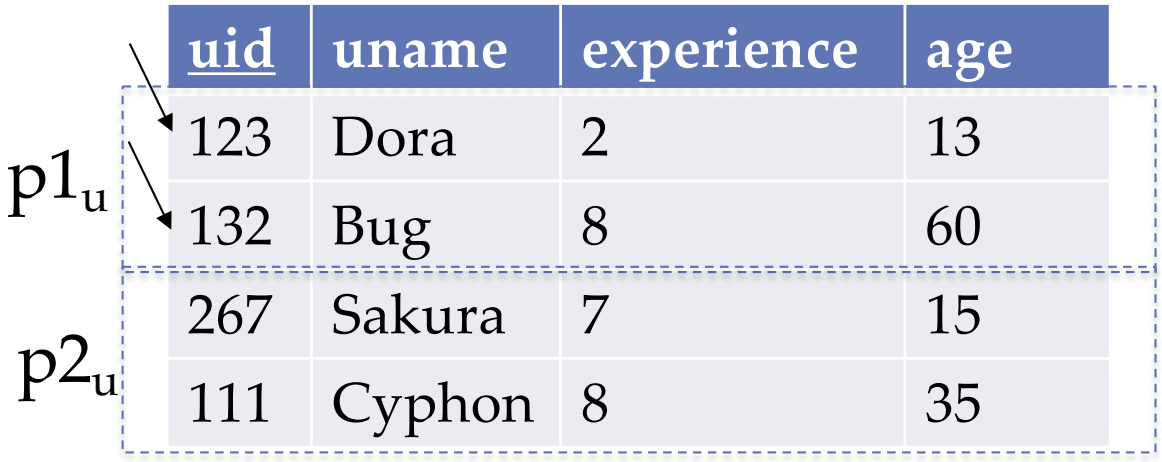
Cardinality Estimation

- $|\ \text{Users} \bowtie \sigma_{(\text{stars} > 3)}(\text{GroupMembers})\ | = ?$
  - Result: $|\ \sigma_{(\text{stars} > 3)}(\text{GroupMembers})\ |$
  - Assuming 1-5 stars, uniform distribution for stars
  - $\text{Red}(\sigma_{(\text{stars} > 3)}(\text{GroupMembers})) = 0.4$
  - Result: 0.4 * |GroupMembers|
- $|\ \sigma_{(\text{experience} > 5)}(\text{Users}) \bowtie (\text{GroupMembers})\ | = ?$
  - Assume 1-10 experience levels, uniform distribution for experience, and uid attribute of GroupMembers also uniformly distributed
  - $\text{Red}(\sigma_{(\text{experience} > 5)}(\text{Users})) = 1/2$
  - Result: ½ * |GroupMembers|

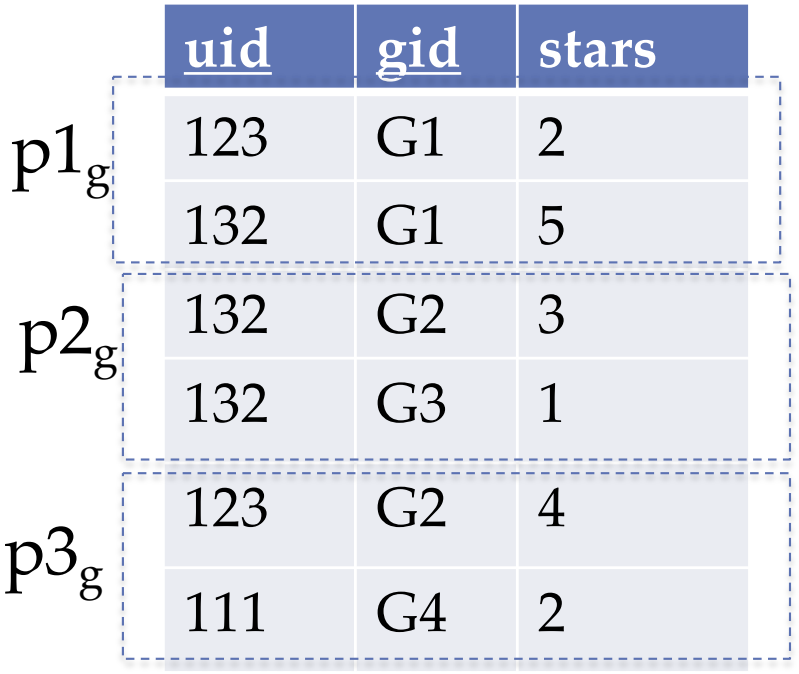
Simple Nested Loop Join

- For each tuple in the *outer* relation Users U we scan the entire *inner* relation GroupMembers GM.

```
foreach tuple u in U do
 foreach tuple g in GM do
    if u.uid == g.uid  then add <u, g> to result
```



	uid	uname	experience	age
$p1_u$	123	Dora	2	13
	132	Bug	8	60
$p2_u$	267	Sakura	7	15
	111	Cyphon	8	35



	uid	gid	stars
$p1_g$	123	G1	2
	132	G1	5
$p2_g$	132	G2	3
	132	G3	1
$p3_g$	123	G2	4
	111	G4	2

Simple Nested Loop Join

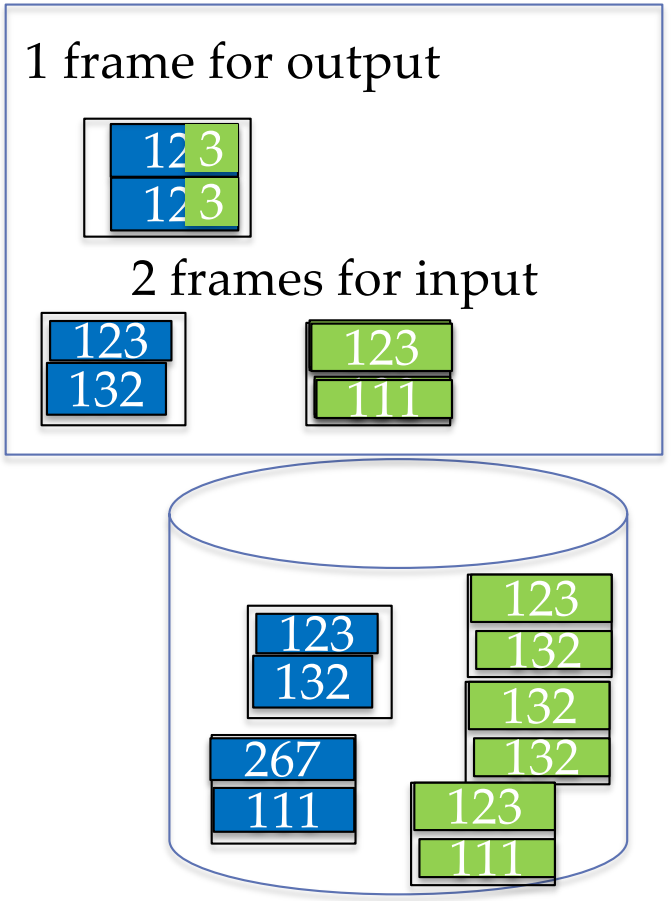
- For each tuple in the *outer* relation Users U we scan the entire *inner* relation GroupMembers GM.

```
foreach tuple u in U do
 foreach tuple g in GM do
    if u.uid == g.uid
       then add <u, g> to result
```

- I/O Cost: OuterPages + CARD(OuterRelation) * InnerPages

  = 500 + 40,000*1000 !

- NOT GOOD
- We need page-oriented algorithm!
- BUT: only 3 memory frames needed



Page Nested Loop Join

- For each *page* p_u of Users U, get each *page* p_g of GroupMembers GM
  - write out matching pairs <u, g>, where u is in p_u and g is in p_g.

```
For each page p_u of Users U
   for each page p_g of GroupMembers GM
      for each tuple u in p_u do
         for each tuple g in p_g do
            if u.uid == g.uid  then add <u, g> to result
```

$p1_u$

uid	uname	experience	age
123	Dora	2	13
132	Bug	8	60

$p2_u$

267	Sakura	7	15
111	Cyphon	8	35

$p1_g$

uid	gid	stars
123	G1	2
132	G1	5

$p2_g$

132	G2	3
132	G3	1

$p3_g$

123	G2	4
111	G4	2

Page Nested Loop Join

- For each *page* p_u of Users U, get each *page* p_g of GroupMembers GM
  - write out matching pairs <u, g>, where u is in p_u and g is in p_g.

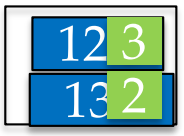
```
For each page p_u of Users U
  for each page p_g of GroupMembers GM
    for each tuple u in p_u do
      for each tuple g in p_g do
        if u.uid == g.uid  then add <u, g> to result
```

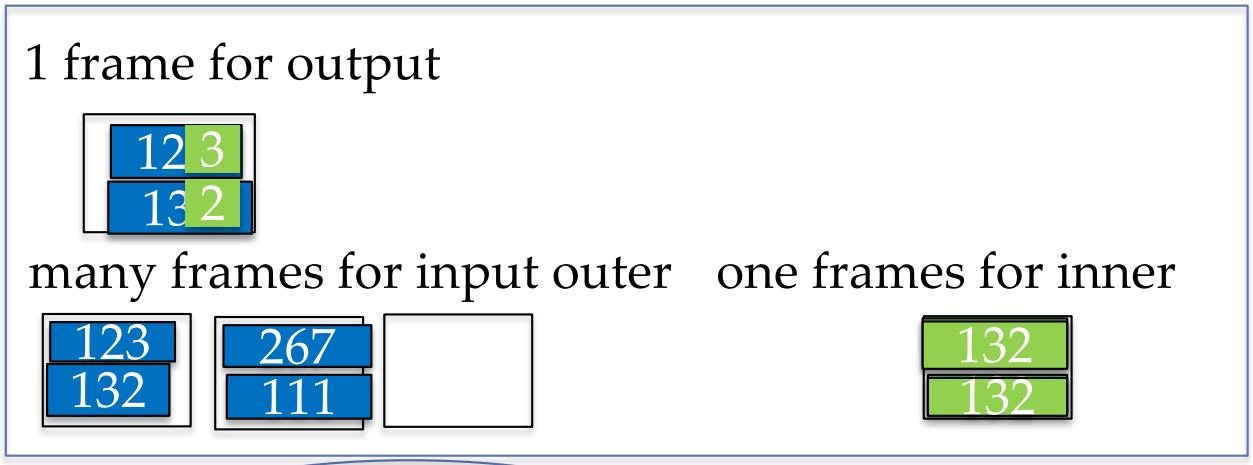
- I/O Cost: OuterPages + OuterPages * InnerPages =

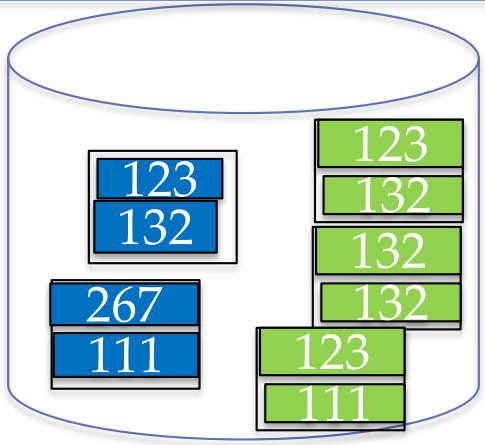
  500 + 500*1000 = 500,500

- Again only 3 memory frames needed
- BUT: main memory is quite large, what if we use more frames?

Block Nested Loop



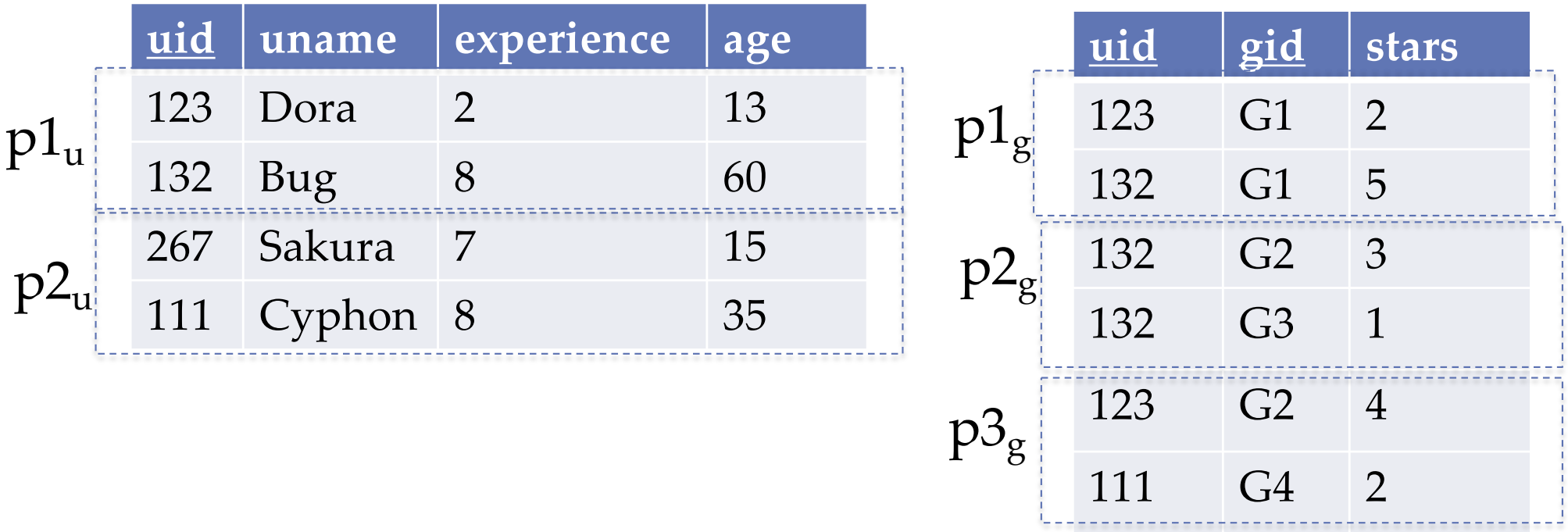




Block Nested Loop Join

- For each *block of pages* bp_u of Users U, get each *page* p_g of GroupMembers GM
  - write out matching pairs <u, g>, where u is in bp_u and g is in p_g.
- *block of pages* bp_u and one page of GM must fit in main memory
  - For each block of pages bp_u
    - Load block into main memory
    - Get first page from GM
      - Do all the matching between users in bp_u and group members in first page
    - Get next page from GM (into the same frame the first one was in before)
      - Do all the the matching between users in bp_u and group members in second page
    - …
- I/O Cost: OuterPages + OuterPages / blocksize * InnerPages
- Memory requirements: bp_u + 1 input frames, 1 output frame

Block Nested Loop



Cost Block Nested Loop Join

- I/O Cost for Example depends on available main memory:
- 52 Buffer Frames:
  - 500 + 500/50 * 1000 = 500 + 10,000
- 502 Buffer Frames
  - 500 + 500/500 * 1000 = 500 + 1000
  - Special case: outer relation fits into main memory!!

- For your back-of-the-envelop calculations you can:
  - ignore the input frame for the outer relation
  - Ignore the output frame

Index Nested Loops Join

- For each tuple in the *outer* relation Users U we find the matching tuples in GroupMembers GM through an index
  - Condition: GM must have an index on the join attribute

```
foreach tuple u in U do
 find all matching tuples g in GM through index
   then add all <u, g> to result
```

	uid	uname	experience	age
$p1_u$	123	Dora	2	13
	132	Bug	8	60
$p2_u$	267	Sakura	7	15
	111	Cyphon	8	35

	uid	gid	stars
$p1_g$	123	G1	2
	132	G1	5
$p2_g$	132	G2	3
	132	G3	1
$p3_g$	123	G2	4
	111	G4	2

Index Nested Loops Join

```
foreach tuple u in U do
 find all matching tuples g in GM through index
   then add all <u, g> to result
```

- Index MUST be on the inner relation (in this case GM).
- I/O Cost: OuterPages + CARD(OuterRelation) * cost of finding matching tuples in inner relation
- Our example
  - 2.5 GM tuples on average per User tuple
- Index on uid on GM is clustered:
  - 500 + 40.000 * (1 leaf page +1 data pages)
- Index on uid on GM is not clustered:
  - 500 + 40.000 * (1 leaf page + 2.5 data pages)

- Main memory requirements?

Index Nested Loops Join

- Switch inner and outer if index is on uid of Users
- Note: uid is primary key in User
  - Only one tuple matches!

```
foreach tuple g in GM do
 find the one matching tuple u in U through index
   then add <g, u> to result
```

- I/O Cost: 1000 + 100.000 * (1 leaf page + 1 data page)

$p1_g$

uid	gid	stars
123	G1	2
132	G1	5

$p2_g$

uid	gid	stars
132	G2	3
132	G3	1

$p3_g$

uid	gid	stars
123	G2	4
111	G4	2

$p1_u$

uid	uname	experience	age
123	Dora	2	13
132	Bug	8	60

$p2_u$

uid	uname	experience	age
267	Sakura	7	15
111	Cyphon	8	35

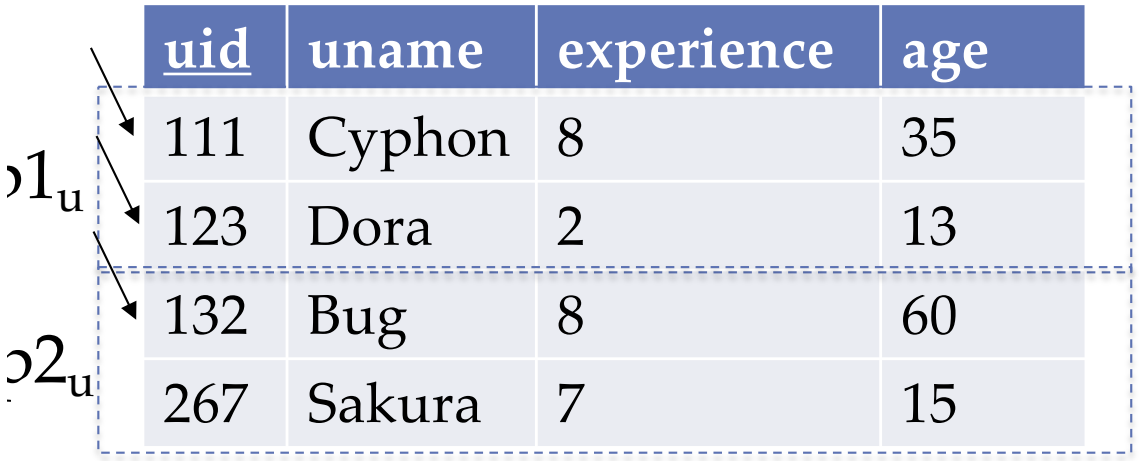
Block Nested Loop vs. Index

- Block nested loop join:
  - OuterPages + OuterPages / (Block Number) * InnerPages
  - Best Case for Block Nested Loop (if outer relation fits in main memory, OuterPages < B)
    - OuterPages + InnerPages
- Index Nested Loop:
  - OuterPages + Card(Outer) * matching tuples Inner
- Index Nested Loop wins if:
  - Card(Outer) is very small
- Example:
  - $\sigma_{(\text{uname} = \text{'Sakura'})}$ (Users) $\bowtie$ (GroupMembers)
  - First select qualifying users; assume only 3 whose name is Sakura; they all fit on same page; this page is outer relation for the join…

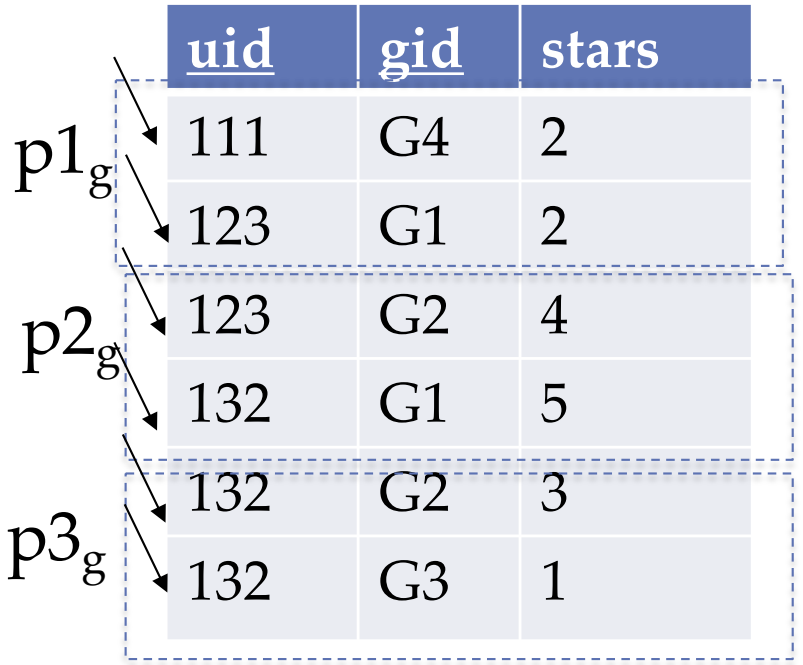
Sort-Merge Join

- Sort U and GM on the join column uid, then scan them to do a "merge" (on join col.), and output result tuples.
- Merge
  - u, g the first tuples in sorted U and GM
  - Repeat until one of U and GM is exhausted
    - If u.uid = g.uid
      - Output matching tuple
      - g = next tuple in GM
    - Else If u.uid > g.uid
      - g = next tuple in GM
    - Else if u.uid < g.uid
      - u = next tuple in U

Example of Sort-Merge Join



uid	uname	experience	age
111	Cyphon	8	35
123	Dora	2	13
132	Bug	8	60
267	Sakura	7	15



uid	gid	stars
111	G4	2
123	G1	2
123	G2	4
132	G1	5
132	G2	3
132	G3	1

Cost of Sort-Merge Join

- Relations are already sorted:
  - UserPages + GroupMemberPages = 500 + 1000
- Relations need to be sorted – simple way:
  - Sort relations and write sorted relations to temporary stable storage
  - Read in sorted relations and merge
  - I/O Costs: assuming 100 buffer pages
    - both Users and GroupMembers can be sorted in 2 passes (Pass 0 and 1): 4* UserPages + 4 GroupPages
    - Final merge: 500 + 1000 = (UserPages + GroupPages)
    - Total: 5 * UserPages + 5 GroupPages

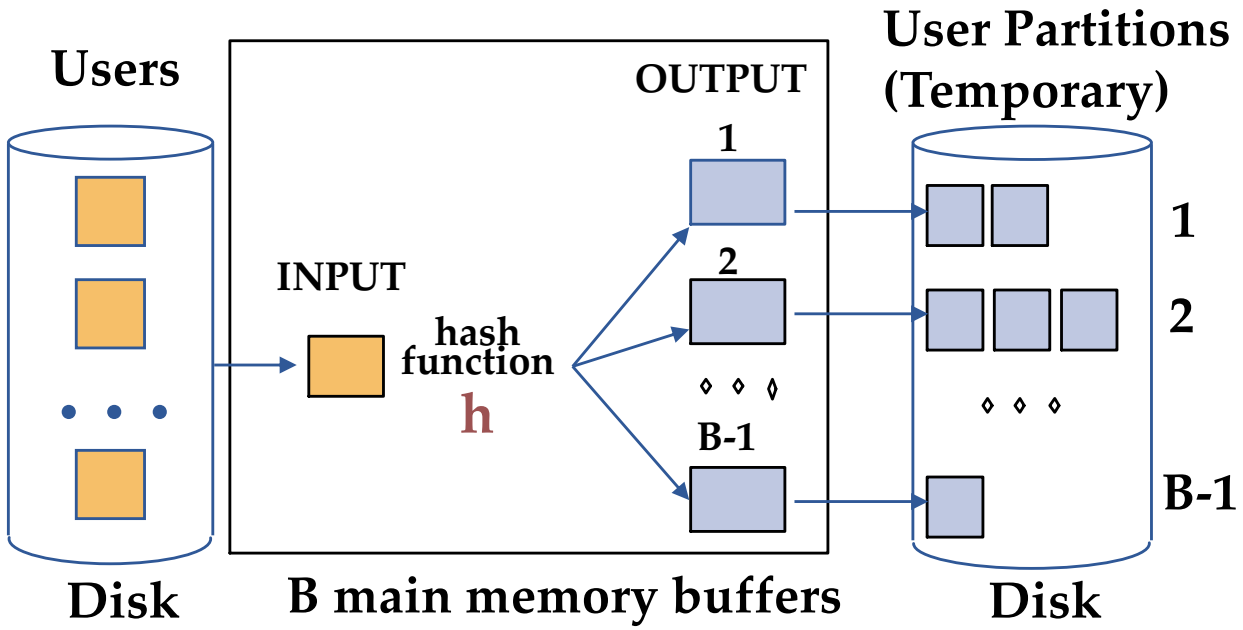
Cost of Sort-Merge Join

- Relations need to be sorted – use pipelining to combine last sort pass and join
  - Sorting performs Pass 0 reads and writes each of the relations: 2 * UserPages + 2 GroupPages
  - Pass 1 reads data, sorts and then performs merge in pipeline fashion (ignore details): UserPages + GroupPages
  - Total: 3 * UserPages + 3 * GroupPages = 4,500

Hash Join: first step

- Partition both relations using hash fn **h** (that returns a value between 1 and B-1 (if there are B buffer frames):
  - h(u.uid) = i → tuple u of User is in UserPartition i.
  - h(g.uid) = i → tuple g of GroupMember is in GroupMemberPartition i.
- Partitioning algorithm for relation U

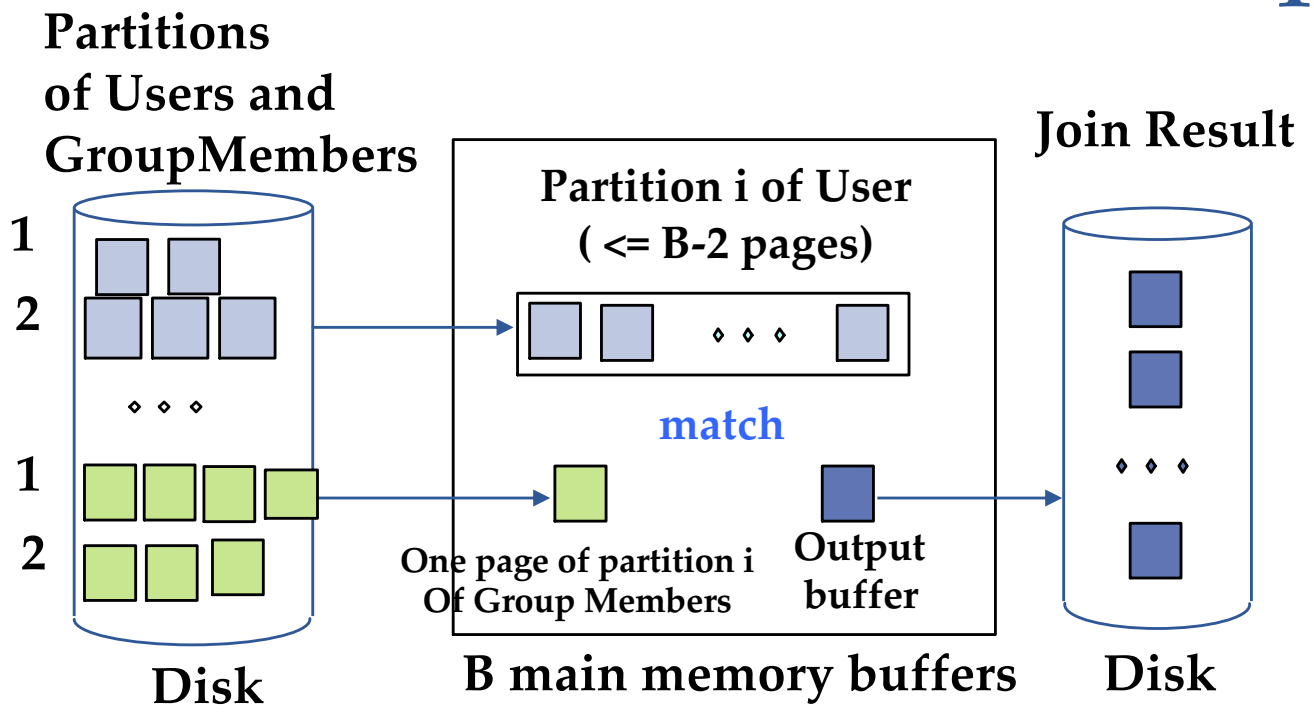
```
For each page of Users U
    for each tuple u in page do
        append u to UserPartition h(u.uid)
```



Hash Join: assumptions

- Assumption for this course:
  - Each partition of User fits into main memory and there are still 2 more buffer frames available
    - Each partition of User smaller than B-2
  - This holds if
    - Hash function creates partitions of equal size
      - Partition size for Users is UserPages / (B-1) = 500 / B-1
      - Partition size for GroupMembers is GroupMemberPages / (B-1) = 1000 / B-1
    - 500 / B-1 (rounded up) <= B -2
      - That is, buffer has at least 24 buffer frames (B=24).

Hash-Join: second step



- For u of User, g of GroupMember
  - u.uid == g.uid and u in UserPartition i ➔ g in GroupMemberPartition i
- Thus, for each i, join UserPartition i with GroupMemberPartition i only.

```
For each i, 1 < … i < ... Number of partitions
   Load partition i of Users into main memory
   For each page of partition i of GroupMembers
      Load page into main memory
      write all matching tuples (u,g) with u.uid = g.uid to output
```

Hash-Join Variation: Main-memory Hash join

- One relation completely fits into main memory.
  - Assume U fits into main memory

```
Read in U
Hash all tuples
For each page I of UG
     Read in page I
     hash each tuple on I and match with U tuple
     write all matching tuples (u,g) with u.uid = g.uid to output
```

Total: UserPages + GroupPages = 1,500

➔Same as block-nested loop join

➔Difference is in how the in main-memory matching is done!